ST207 Assignment 1 – E-Commerce Database Report

Candidate number: 59849

Question 1: Conceptual / E-R Model Explanation

The E-R design shows the main entities, how they are connected and the rules that govern the data. It describes a minimal single-retailer online shop, similar to a fashion or electronics store, and is limited to what the frequent use cases need: browsing a catalogue, keeping a basket, placing and paying for orders, and reviewing products. Marketplace features such as sellers, promotions and returns were left out on purpose, because they add entities without changing how the core processes work.

Customer:

- Each customer is identified by customer_id, an ID with no meaning of its own, so an email address can change without breaking any link. The stored details are first_name, last_name, email and registered_at.
- A customer can hold 0..N Address entries for delivery and billing.
- A customer can own 0..1 Cart and place 0..N Customer Orders, recording activity before and after checkout.
- A customer can write 0..N Review records about products.

Address:

- An address is identified by address_id and records street, city, postcode, country and an is_active flag. The flag lets an old address be retired without deleting it, so orders that used it keep their delivery history.
- An address is owned by one customer only.

Category:

- Categories are identified by category_id and have a name. A recursive "parent of" relationship (0..1 parent, 0..N children) builds a tree such as Electronics > Audio > Headphones.
- A category groups 0..N products; each product has exactly one category, and a category can be empty.

Product and Product Variant:

- A Product is identified by product_id and describes an item as shown in the catalogue. It stores name, brand and description, and has a derived attribute, avg_rating, which is calculated when needed and not stored.
- A Product Variant is what is actually sold, identified by variant_id. It stores sku, unit_price, stock_quantity and the optional size and colour. Keeping price and stock here lets one colour or size sell out without affecting the others.
- A product has 1..N variants, while a variant refers to exactly one product.
- size and colour are optional (O) because many products, such as a kettle or a charger, have neither. NULL therefore means "not applicable".

Cart and Cart Item:

- A Cart holds a customer's temporary selection before checkout and records created_at. A customer owns 0..1 Cart, so there is one persistent basket per customer.
- Cart Item resolves the many-to-many link between Cart and Product Variant. It stores quantity and created_at, which allows abandoned carts to be dated.

Customer Order and Order Item:

- An order records order_date, status and shipping_fee, with an optional shipped_at that stays NULL until dispatch. order_total is a derived attribute and is not stored.
- Each order ships to exactly one Address and is billed to exactly one Address, modelled as two separate relationships. An address can be used by 0..N orders.
- An order contains 1..N Order Items, so it cannot be empty. Order Item links an order to a Product Variant and stores quantity and unit_price_at_purchase, a price snapshot that keeps past orders correct when prices change.

Payment and Card Payment:

- An order is settled by 0..N Payment entries. Zero is allowed because a new order may not have been paid yet, and several are allowed because a payment can fail and be retried, or the total can be split across methods. Payment stores amount, method, status, attempted_at and transaction_ref.
- Card Payment is a specialisation of Payment. Only card payments have card_network and card_last4, so other payments do not carry empty card columns. The specialisation is partial because not every payment is a card payment, and disjoint because a payment cannot be of two types (trivially, with only one subtype).

Review:

- Review is identified by review_id and stores rating (1 to 5), review_date and an optional comment. A customer can write 0..N reviews and a product can receive 0..N reviews.

Optionality and assumptions:

- A cardinality of 1 (mandatory) is used where a record cannot exist without its parent, for example Address to Customer, Product Variant to Product, Order Item to Customer Order and Customer Order to Address. A cardinality of 0 (optional) is used where there may be nothing yet, such as a customer with no orders or a product with no reviews.
- A cancelled order keeps its rows. Its status changes instead of the record being deleted, so financial and stock history stay traceable.
- Reviews are assumed to come from customers who bought the product. The model does not link a review to an order line, so this is an assumption about the data and not something the database enforces. A customer can review a product only once, which is enforced.

Conclusion:

The model covers the main activities of an online shop: customers, the catalogue, carts, orders, payments and reviews. It supports the use cases in the SQL section, and I checked the model against the scenario by confirming that each use case can be answered from its entities and relationships. The E-R diagram is attached as ER_Model_FINAL.png and shows every cardinality in Crow's foot notation.

Question 2: Relational Model Explanation

The relational design turns the E-R model into 12 linked base tables and three views. Entities become tables, attributes become columns and relationships are shown with keys, with some adjustments made along the way. The relational diagram (Relational_Model_FINAL.png) shows the tables, primary keys and foreign keys, and the data types are stored in its ERDPlus source file (Relational_Model_FINAL.erdplus). The views were added when writing the DDL.

Adaptations from the Conceptual / E-R Model:

- Each entity becomes a base table, and its attributes become the columns of that table.
- Many-to-many relationships become linking tables:
  - CartItem links Cart with ProductVariant and stores quantity and created_at.
  - OrderItem links CustomerOrder with ProductVariant and stores quantity and unit_price_at_purchase.
  - UNIQUE(cart_id, variant_id) and UNIQUE(order_id, variant_id) stop the same variant appearing twice in one cart or order, so a repeat becomes a change of quantity.
- The 1:0..1 relationship between Customer and Cart is implemented by customer_id in Cart with a UNIQUE constraint.
- The parent relationship in Category becomes a foreign key, parent_category_id, that points to another row of the same table. It can be NULL, which marks a top-level category.
- Card payments get their own table, CardPayment, which uses payment_id as both its primary key and a foreign key to Payment. I did not put the card details in Payment because card_network and card_last4 would be empty for PayPal and bank transfers.
- CustomerOrder holds two foreign keys to Address, shipping_address_id and billing_address_id, for the two conceptual relationships.
- The order table is named CustomerOrder because ORDER is a reserved word in SQL.

- The two derived attributes, order_total and avg_rating, are not stored. They are implemented as views.

Views:

- v_order_total is built from CustomerOrder and OrderItem (LEFT JOIN). Its columns are order_id, customer_id, order_date, status, items_total, shipping_fee and order_total, where order_total is the sum of quantity $\times$ unit_price_at_purchase plus shipping_fee. It keeps orders that have no lines.
- v_product_rating is built from Product and Review (LEFT JOIN). Its columns are product_id, name, category_id, review_count and avg_rating. It keeps products with no reviews.
- v_order_payment is built from v_order_total and Payment (completed payments only). Its columns are order_id, order_total and paid_total, so each order's total can be compared with the completed payments made for it. It is used by a validation query.
- Views store no data, so the derived values can never become outdated, and the join logic is written once and reused.

Implementation of Relationships and Keys:

- Foreign keys link each child table to its parent in the one-to-many relationships. For example:
  - Address, Cart and CustomerOrder use customer_id to reference Customer.
  - ProductVariant uses product_id to reference Product, and Product uses category_id to reference Category.
  - Payment uses order_id to reference CustomerOrder, and Review uses customer_id and product_id.
- CustomerOrder references Customer, and the shipping and billing Address, through simple foreign keys. That an order uses addresses of its own customer is a data rule, checked with a validation query.
- The minimum and maximum cardinalities are enforced with keys and other rules. An attribute not marked (O) must have a value, and mandatory participation is a foreign key that cannot be NULL. The only foreign key that can be NULL is parent_category_id.
- UNIQUE(customer_id, product_id) on Review gives one review per customer per product. CHECK constraints limit rating to 1..5, require positive prices and quantities, keep stock_quantity at zero or above, and restrict status, method and card_network to fixed lists. The triggers enforce the order status flow and the stock rules.
- Two rules cannot be written as simple constraints in SQLite: an order must have at least one item (1..N), and every card payment must have a CardPayment row. Both are kept by the order in which the data is loaded and are checked with validation queries.
- The delete rules follow ownership. Carts, cart items and order lines are removed together with the record they belong to (CASCADE). Records that others depend on, such as customers, orders, products and addresses, cannot be deleted while those dependants exist (RESTRICT), so financial history is never lost by accident.

Other design decisions:

- The tables follow third normal form: each column describes only the key of its own table, so no fact is stored twice. The one exception is Address, where city and country are stored next to the postcode as a deliberate simplification.
- Two columns repeat information on purpose. unit_price_at_purchase keeps the price at the time of sale, so old orders stay correct when prices change, and stock_quantity is a stored running balance that the triggers keep up to date.

Conclusion:

The relational design turns the E-R model into a usable database structure. Linking tables handle the many-to-many associations, the subtype table handles the specialisation and views replace the derived attributes. Together, these choices keep the data consistent and support the project requirements.

Question 3: Database Creation

Creation order:

- Foundations: Create Customer and Category as the independent starting tables.
- Customer and product tables: Build Address and Cart, which depend on Customer, and Product, which depends on Category.
- Variants and carts: Create ProductVariant, followed by CartItem, which connects carts and variants through their many-to-many association.

- Orders: Create CustomerOrder, which references Customer and Address, followed by OrderItem, which connects orders to their purchased variants.
- Payments: Create Payment before CardPayment, which reuses payment_id as its primary key.
- Remaining tables: Add Review, then the indexes on foreign keys and the three views, which can only be created once the tables they read exist.

Foreign key handling:

- No foreign key had to be relaxed or added afterwards with ALTER TABLE, because no tables depend on each other in a circle. The only table that refers to itself is Category (parent_category_id), and top-level categories have no parent, so they are inserted first and their children afterwards.
- Tables are created parent first, so every table that a foreign key points to exists before the table that uses it. The data was loaded in the same order.
- SQLite only checks foreign keys when PRAGMA foreign_keys = ON is set, so this is the first line of the schema, data, trigger, query and validation scripts. After loading, PRAGMA foreign_key_check returned no rows.

Constraints and implementation of data types:

- NOT NULL is applied to every column except the optional attributes. UNIQUE covers email, sku, transaction_ ref, one review per customer and product, one line per variant in each cart and order, and customer_id in Cart, which implements the one-to-one link between Customer and Cart.
- The types chosen in the relational model (INT, VARCHAR, CHAR, DATE and NUMERIC(10,2)) are implemented in SQLite as follows: INT becomes INTEGER, VARCHAR and CHAR become TEXT, and NUMERIC(10,2) becomes DECIMAL(10,2), which SQLite stores as a number. SQLite has no DATE type, so dates are stored as text in the format YYYY-MM-DD HH:MM:SS, which sorts and compares correctly.
- CHECK conditions reject invalid values, for example an order status must be Pending, Paid, Shipped, Delivered or Cancelled, and a rating must be between 1 and 5.
- The triggers were created after the data was loaded, because the loaded stock_quantity already reflects the past sales and the triggers should not change it. Two log tables, inventory_log and order_status_log, are created with the triggers. They record changes and are not part of the business model.

Question 4: Data Sources

- All sample records were generated synthetically by a Python script (Synthetic_Data_Generator_FINAL.py) written with help from Claude; no external dataset was imported. The script uses a fixed random seed (42), so it reproduces the same data file every time. The script is submitted with this report.
- The generated sample records cover:
  - 19 categories, 46 products and 169 variants, written by hand with realistic names, brands and prices, spread across the top-level categories as Electronics 19, Fashion 12, Sports & Outdoors 8, Home & Kitchen 7 and Toys & Games 0, which is empty on purpose.
  - 260 customers with 388 addresses. Each customer was given a behaviour profile: heavy buyers, regular buyers, occasional buyers and customers who never order.
  - 916 orders with 1,501 order lines, with a November and December peak, a discount in late November and fifteen co-purchased product pairs built in for basket analysis.
  - 1,069 payments (712 by card), including orders with a failed attempt followed by a successful retry (118), orders split across two methods (35) and refunds after cancellation (41).
  - 122 carts with 305 cart items, some of them variants that the customer later bought anyway.
  - 369 reviews of delivered purchases, with four products left unreviewed on purpose.
- I loaded the records in the order of the foreign key dependencies, so parent records exist before the records that refer to them. Order status follows order age, so the most recent orders are Pending or Paid and older orders are mostly Delivered, with some Cancelled. stock_quantity was generated after all the sales, so it is consistent with the order history and never negative.
- I checked the data with sixteen validation queries. Each one looks for a problem, and all of them return no rows: broken foreign keys; orders sent to another customer's address; paid, shipped or delivered orders not covered by completed payments; orders without items; shipped_at that does not match the order status; card payments without a CardPayment row; reviews with no earlier delivered purchase of that product; orders placed before the customer registered; loops in the category tree; refunds on orders that were not cancelled; duplicate emails, SKUs

or payment references; the same variant twice in one cart or order; more than one review per customer and product; and invalid values (ratings outside 1 to 5, quantities or prices that are not positive, unknown statuses or methods).

- The same file also shows that the data includes the cases the use cases need: 49 customers with no orders, four unreviewed products, three out-of-stock variants, one empty category and 50 orders billed to a different address from the shipping address.

Question 5: SQL Use Cases

The three use cases cover customer accounts, sales performance and marketing analytics, and each contains two queries, numbered 5.1 to 5.6. All queries use only constructs taught in the course (ST207, 2026), and the window functions follow the SQLite Tutorial (n.d.). SQLite has no CREATE FUNCTION, so I did not use user-defined functions; the views are used instead for reusable logic. Cancelled orders are excluded from the analytical figures.

Question 5, Use Case 1: Customer details, orders and abandoned carts:

Customer service staff use Query 5.1 to look up customers and their orders, and marketing uses Query 5.2 to win back customers with abandoned carts. Query 5.1 brings together customer details and their orders. I first selected the fields required for the report, then addressed the problem of customers holding several addresses and several orders. The CTE main_address picks the lowest-numbered active address of each customer, so every customer appears once. The CTE order_stats totals the non-cancelled orders from the v_order_total view, giving the number of orders, the total spent and the date of the last order. A CASE expression replaces a missing total with 0, giving the reader a meaningful value instead of NULL. A correlated subquery counts the failed payment attempts of each customer, and the window function RANK() OVER (PARTITION BY country ORDER BY total_spent DESC) ranks each customer's spend within their country. Query 5.2 lists abandoned carts for a marketing email. A WHERE condition keeps cart items that have been idle for more than seven days, and NOT EXISTS removes customers who have since bought the same variant. A HAVING condition keeps carts worth at least £500, and a CASE expression assigns a suggested action.

Customer is the starting table of Query 5.1 because the result must include everyone, even customers without an order. LEFT OUTER JOINs keep those customers while adding their address and order information. In Query 5.2, I grouped the rows by customer, so each customer's cart items become one row with a count and a total value. The output shows that customer 135 is the top spender in the United Kingdom with £6,585.92 from 7 orders, while customer 102 placed 13 orders for £5,573.03. The top customer in Ireland has not ordered for 245 days, and 51 customers have no non-cancelled order (49 have no order at all and 2 have only cancelled ones). In Query 5.2, 19 customers have at least £500 waiting in an idle cart, the largest being £3,201.99. The resulting outputs are presented below:

	customer_id	customer	city	country	n_orders	total_spent	days_since_last	failed_payments	rank_in_country
1	135	Ravi Garcia	Liverpool	United Kingdom	7	6585.92	29.0	1	1
2	181	Amelia Hill	London	United Kingdom	11	6239.75	7.0	2	2
3	120	Mia Ahmed	London	United Kingdom	7	6085.9	81.0	0	3
4	102	Mia Kowalski	Manchester	United Kingdom	13	5573.03	7.0	1	4
5	2	Sophie Hall	Birmingham	United Kingdom	11	5013.91	159.0	1	5
5	75	Isla Patel	Cardiff	United Kingdom	15	4931.57	168.0	1	6
7	116	Jack Brown	London	United Kingdom	6	4135.71	51.0	1	7
3	148	Jack Smith	London	United Kingdom	14	3840.12	7.0	2	8

Query 5.1: customer details and orders (first rows; the email and last-order-date columns are hidden and two column names shortened to fit)

	customer_id	customer	email	abandoned_items	cart_value	days_idle	action
1	161	George Robinson	george.robinson@example.com	3	3201.99	10.0	HIGH value - send offer
2	52	Grace Patel	grace.patel@example.com	2	1747.98	13.0	HIGH value - send offer
3	139	Grace Davies	grace.davies@example.com	3	1642.99	8.0	HIGH value - send offer
4	68	Emily Thompson	emily.thompson@example.com	2	1608.99	12.0	HIGH value - send offer
5	111	Anna Brown	anna.brown@example.com	2	1598.99	12.0	HIGH value - send offer
6	220	Omar Ward	omar.ward@example.com	2	1528.99	18.0	HIGH value - send offer
7	75	Isla Patel	isla.patel@example.com	4	1402.98	18.0	MEDIUM value - reminder
8	57	Oscar Turner	oscar.turner@example.com	2	1396.0	17.0	MEDIUM value - reminder

Query 5.2: abandoned-cart recovery list

Question 5, Use Case 2: Sales performance (product and monthly revenue):

Merchandising and management use these queries to see which products and months earn the most. Query 5.3 finds the three highest-revenue products within each top-level category. I extended it with the share of category revenue and the average rating to provide further context. The recursive CTE root_of maps every category, at any depth, to its top-level category, so Headphones is counted under Electronics. The CTE product_revenue totals quantity multiplied by unit_price_at_purchase for each product, which uses the price actually paid rather than the current list price. A window function, SUM() OVER (PARTITION BY category), gives the category total, from which each product's share of its category's revenue is calculated, and RANK() OVER (PARTITION BY category ORDER BY revenue DESC) places higher-revenue products first. The final SELECT keeps only the rows where the rank is no greater than three. Query 5.4 reports monthly revenue. A recursive CTE generates every month, and LEAD finds the start of the following month so that orders can be matched by date range. LAG calculates the month-on-month growth, AVG() OVER (ROWS BETWEEN 2 PRECEDING AND CURRENT ROW) calculates a three-month moving average, and RANK orders the months by revenue.

The view v_product_rating supplies the average rating in Query 5.3, and products without reviews show NULL. The window functions in Query 5.4 run over the whole history, while only the last twelve complete months are displayed. The output shows that the Zenith Pro 15 Laptop brings in £53,681.68, which is 34.3% of Electronics revenue, and the AirCrisp Air Fryer accounts for 45.3% of Home & Kitchen revenue; both are rated lower than the other products in their top three. Revenue peaks in November 2025 at £15,271.67, and December has the most orders (71) but the lowest average order value (£191.91), consistent with the late-November discounts. The resulting outputs are presented below:

	category	rank_in_category	product	revenue	units_sold	pct_of_category	avg_rating
1	Electronics	1	Zenith Pro 15 Laptop	53681.68	39	34.3	3.64
2	Electronics	2	Ember Student Laptop	21863.68	48	14.0	4.0
3	Electronics	3	Nova X5 Smartphone	18245.34	39	11.7	4.38
4	Fashion	1	Trail Runner Limited Edition	7953.58	74	25.8	NULL
5	Fashion	2	Leather Chelsea Boots	7298.98	63	23.7	4.45
6	Fashion	3	Waterproof Jacket	3027.42	35	9.8	4.25
7	Home & Kitchen	1	AirCrisp Air Fryer 5L	6320.15	65	45.3	3.44
8	Home & Kitchen	2	CloudSoft Duvet	2471.76	45	17.7	4.5

Query 5.3: top 3 products by revenue in each category (first 8 rows)

	month_start	revenue	orders	customers	avg_order_value	mom_growth_pct	moving_avg_3m	revenue_rank_all_time
1	2025-10-01	8557.12	30	27	285.24	34.9	7612.49	12
2	2025-11-01	15271.67	56	47	272.71	78.5	10057.41	1
3	2025-12-01	13625.32	71	59	191.91	-10.8	12484.7	3
4	2026-01-01	9027.59	33	28	273.56	-33.7	12641.53	11
5	2026-02-01	8074.53	35	33	230.7	-10.6	10242.48	13
6	2026-03-01	14334.54	36	33	398.18	77.5	10478.89	2
7	2026-04-01	13530.45	47	42	287.88	-5.6	11979.84	4
8	2026-05-01	12855.67	36	35	357.1	-5.0	13573.55	5

Query 5.4: monthly revenue trend (first 8 of the 12 months shown)

Question 5, Use Case 3: Marketing analytics:

Marketing uses these queries to decide which products to bundle and which customers to target. Query 5.5 analyses which products are bought together. I first created a CTE holding the distinct pairs of order and product from valid orders. A self-join on order_id, with a.product_id < b.product_id, produces each pair once, and HAVING COUNT(*) >= 8 removes rare pairs. For each pair, the query calculates support (the share of all orders containing both products), confidence (the share of orders with the first product that also contain the second) and lift, where lift compares how often the pair is bought together with how often we would expect if the two products were bought independently. Query 5.6 segments customers by recency, frequency and monetary value (RFM). A LEFT OUTER JOIN keeps customers who never ordered, NTILE(4) scores each measure from 1 to 4, and a CASE expression maps the scores to six segments. A scalar subquery gives the share of total revenue for each segment.

The strongest pair in Query 5.5 is the Classic Cotton T-Shirt with the Slim Fit Chinos, bought together in 21 orders with a lift of 13.6. The 2-Person Tent and the Sleeping Bag are bought together most often (28 orders) but have a lower lift of 7.4, because both sell well alone. In Query 5.6, the 64 customers in the Champions segment produce 59.5% of revenue, and the 27 customers in the At risk big spenders segment produce 23.1% but have not ordered for an average of 224 days. The 1 to 4 scores are relative to the current customers: a score of 4 means the top quarter. The resulting outputs are presented below:

	product_A	product_B	pair_orders	support_pct	confidence_A_to_B_pct	lift
1	Classic Cotton T-Shirt	Slim Fit Chinos	21	2.5	42.0	13.6
2	Zenith Pro 15 Laptop	USB-C Hub 7-in-1	12	1.43	35.3	12.4
3	CloudSoft Duvet	Egyptian Cotton Sheet Set	18	2.14	50.0	12.0
4	Nova X5 Smartphone	ShieldCase Phone Case	10	1.19	35.7	11.6
5	Pro Yoga Mat	Resistance Bands Set	22	2.62	38.6	9.0
6	2-Person Tent	Sleeping Bag -5C	28	3.33	36.8	7.4
7	Ember Student Laptop	Glide Wireless Mouse	14	1.66	41.2	6.3
8	Glide Wireless Mouse	KeyForge Mechanical Keyboard	11	1.31	20.0	3.9

Query 5.5: products bought together

	segment	customers	avg_spend	avg_orders	avg_days_since_last	segment_revenue	revenue_share_pct
1	1 Champions	64	2081.59	7.2	55.0	133221.96	59.5
2	2 At risk big spenders	27	1911.8	5.7	224.0	51618.53	23.1
3	3 New / promising	31	278.91	1.6	63.0	8646.35	3.9
4	4 Needs attention	22	720.35	3.3	169.0	15847.76	7.1
5	5 Lost / low value	65	223.52	1.6	356.0	14528.94	6.5
6	6 Never bought	51	0.0	0.0	NULL	0.0	0.0

Query 5.6: RFM segment summary

Question 6: Triggers

Question 6i)

The first set of triggers checks stock before an OrderItem insert is accepted. A BEFORE INSERT trigger rejects a line when the order is not Pending or when the requested quantity exceeds the available stock. This prevents an order line from being added when it cannot be supplied, and stock is reserved only when the order line is created, not when an item is added to a cart. An AFTER INSERT trigger then deducts the units from stock_quantity and records the movement in inventory_log. Further triggers apply the same check when a quantity is increased, adjust the stock when a quantity is changed or a line is deleted, and block the deletion of a line from an order that is already paid or shipped. The checks follow the seminar pattern SELECT CASE WHEN ... THEN RAISE(FAIL, 'ERROR: ...'), and the log table works like emp_log (ST207, 2026).

I tested an acceptable quantity and an excessive quantity on a variant with 3 units in stock. The order of 2 units succeeded: stock fell from 3 to 1 and a row was added to inventory_log. The request for 5 units was rejected with "ERROR: insufficient stock for this product variant" and stock stayed at 1, which confirms that the stock check blocks a quantity above the available amount. The resulting outputs are presented below:

	log_id	variant_id	order_id	change_qty	stock_after	description	logged_at
1	1	139	915	-2	1	Order line placed	2026-11-01 01:01:56

Demo 1A: order of 2 units, logged by the trigger (passes)

```
Execution finished with errors.
Result: ERROR: insufficient stock for this product variant
At line 1:
-- DEMO 1B (VIOLATION): 5 units requested, only 1 left
-- 1B-1  >>> MUST ERROR <<<  message pane: ERROR: insufficient stock for this product variant
INSERT INTO OrderItem (quantity, unit_price_at_purchase, variant_id, order_id) SELECT 5, unit_price, variant_id, 915 FROM
ProductVariant WHERE sku = 'STR-031-0139';
```

Demo 1B: request for 5 units, trigger error (violation)

Question 6ii)

The second set of triggers checks the status of an order against its payments. A BEFORE UPDATE trigger allows only the legal flow: Pending to Paid, Paid to Shipped, Shipped to Delivered, and Pending or Paid to Cancelled. It also refuses Paid unless the order has items and the sum of its completed payments is at least the order total, which is calculated from the order lines and the shipping fee. AFTER UPDATE triggers stamp shipped_at when an order ships, record each change in order_status_log and, on cancellation, return the stock of every line, change completed payments to Refunded and void pending ones. A BEFORE INSERT trigger on Payment rejects a payment for a cancelled order and a completed payment larger than the outstanding balance. The strict status rules are a design choice: returns after delivery would need an additional status and a refund process, which this minimal model leaves out.

I tested order 915, a pending order with no completed payment, and order 910, a paid order. The pending order could not be marked Paid without a completed payment, and the update was rejected with "ERROR: completed payments do not cover the order total". Jumping from Pending to Delivered was rejected as an invalid status transition. In contrast, the paid order moved to Shipped, shipped_at was filled in automatically and the change appeared in order_status_log. I also tested an edge case: cancelling the pending order returned the 2 shoes to stock (from 1 back to 3) and changed its pending payment to Failed. The resulting outputs are presented below:

```
Execution finished with errors.
Result: ERROR: completed payments do not cover the order total
At line 1:
-- ================ TRIGGER 2: ORDER LIFECYCLE AND PAYMENT CONSISTENCY ================
-- DEMO 2A (VIOLATION): order 915 has no COMPLETED payment, so it cannot be marked Paid
-- 2A-1  >>> MUST ERROR <<<  message pane: ERROR: completed payments do not cover the order total
UPDATE CustomerOrder SET status = 'Paid' WHERE order_id = 915;
```

```
Execution finished with errors.
Result: ERROR: invalid order status transition
At line 1:
-- DEMO 2B (VIOLATION): skipping the workflow, Pending -> Delivered
-- 2B-1  >>> MUST ERROR <<<  message pane: ERROR: invalid order status transition
UPDATE CustomerOrder SET status = 'Delivered' WHERE order_id = 915;
```

Demos 2A and 2B: trigger errors (violations)

	log_id	order_id	old_status	new_status	logged_at
1	1	910	Paid	Shipped	2026-11-06 03:04:33

	order_id	status	order_date	shipped_at
1	910	Shipped	2026-10-26 01:20:56	2026-11-06 03:04:33

Demo 2C: order 910 moved from Paid to Shipped, with shipped_at filled in by the trigger and the change logged (passes)

	payment_id	amount	status
1	1068	238	Failed

```
Execution finished without errors.
Result: 1 rows returned in 8ms
At line 6:
-- 2D-3  the pending payment of order 915 is now Failed (voided)
SELECT payment_id, amount, status FROM Payment WHERE order_id = 915;
```

	sku	stock_after_cancel
1	STR-031-0139	3

Demo 2D: cancelling order 915 voids its pending payment and returns the stock to 3

References:

1. ST207 (2026) All lecture slides, seminar materials and lecture and seminar recordings. [online] Available via: ST207 Moodle page (Accessed: 01/11/2026).
2. SQLite Tutorial (n.d.) SQLite Window Functions. URL: https://www.sqlitetutorial.net/sqlite-window-functions/
   - Linked from the Week 4 slides on window functions, which the window-function queries in Question 5 follow.
3. Anthropic (2026) Claude. (Accessed: 01/11/2026).
   - Used to create the synthetic records described under Data Sources, and to check spelling, grammar, report formatting and structure, whether every question was answered and whether each answer had enough detail, with all outputs run, checked and tested by me. The chat log of the report proofreading and review conversation is attached (Claude_Chat_Log_Report_Review_FINAL.pdf).

Statement on the use of generative AI:

I used Claude (Anthropic) to generate the synthetic data for the database (Question 4). I also used Claude to check spelling, punctuation and grammar, to format the report and check its structure, to check that every question in the brief had been answered, and to check that each answer had enough detail. I decided which use cases and queries to include and directed changes to the code and the report, for example matching the lecture and seminar style and reducing the queries to six. I ran and tested all code in DB Browser for SQLite and took every screenshot from my own runs. I have attached the source files wherever the brief requires them, including the script used to generate the synthetic data (Synthetic_Data_Generator_FINAL.py).